



SE3082 - Parallel Computing

Lab-06

IT24101503

A. A. B. Shamim Basha

Exercise 01:

Pragma omp barrier blocks every thread in the parallel region until all threads have reached. Therefore, no thread can print either message after the barrier until every thread has printed its first thread number line and arrived at the barrier. The order of output is otherwise nondeterministic because the threads are scheduled independently.

```
#include <omp.h>
#include <stdio.h>

int main(void) {
#pragma omp parallel
{
    int thread_id = omp_get_thread_num();
    printf("thread # %d\n", thread_id);

#pragma omp barrier
    printf("Outside Barrier # %d\n", omp_get_thread_num());
    printf("Hi again from thread # %d\n", omp_get_thread_num());
}

    return 0;
}
```

Exercise 02:

Pragma omp single selects one thread to execute its structured block. The other threads skip the block. An implicit barrier follows single unless `nowait` is specified, so all threads wait before printing the final message.

```
#include <omp.h>
#include <stdio.h>

int main(void) {
    #pragma omp parallel
    {
        printf("thread # %d\n", omp_get_thread_num());

        #pragma omp single
        {
            printf("Inside Single # %d\n", omp_get_thread_num());
            printf("Exiting Single\n");
        }

        printf("Hi again from thread # %d\n", omp_get_thread_num());
    }

    return 0;
}
```

Exercise 03:

Pragma omp master allows only the master thread (thread 0) to execute its block. Unlike ***single***, it has no implicit barrier at the end, so other threads do not wait for the master block before continuing.

```
#include <omp.h>
#include <stdio.h>

int main(void) {
#pragma omp parallel
{
    printf("thread # %d\n", omp_get_thread_num());

#pragma omp master
    {
        printf("Inside Master # %d\n", omp_get_thread_num());
        printf("Exiting Master\n");
    }

    printf("Hi again from thread # %d\n", omp_get_thread_num());
}

return 0;
}
```

Exercise 04:

- Without collapse, OpenMP distributes only the outer row loop. Each assigned thread executes all ten columns for its rows, so rows are the scheduling units.
- With collapse(2), OpenMP combines the two perfectly nested loops into one logical loop of 50 iterations. Individual (row, column) pairs can then be distributed among threads. The output order is nondeterministic in both cases.

```
#include <omp.h>
#include <stdio.h>

int main() {

#pragma omp parallel for collapse(2)
    for (int r=1; r<=5; r++)
        for (int k=1; k<=10; k++) {
            printf("r = %d, k = %d, thread # = %d\n", r, k, omp_get_thread_num());
        }

    return 0;
}
```

Exercise 05:

```
#include <omp.h>

int fib(int n) {
    int i, j;
    if (n < 2)
        return n;
    else {
        #pragma omp task shared(i)
        i = fib(n-1);

        #pragma omp task shared(j)
        j = fib(n-2);

        #pragma omp taskwait
        return i + j;
    }
}
```

Exercise 06:

```
#include <stdio.h>
#include <stdlib.h>
#include <omp.h>

#define N 1000000
#define STRIP_SIZE 16

int main() {
    float *A = (float*)malloc(N * sizeof(float));
    float *B = (float*)malloc(N * sizeof(float));
    float *C = (float*)malloc(N * sizeof(float));

    for (int i = 0; i < N; i++) {
        A[i] = 1.0f;
        B[i] = 2.0f;
    }

    #pragma omp parallel for
    for (int base = 0; base < N; base += STRIP_SIZE) {
        int limit = base + STRIP_SIZE;
        if (limit > N) limit = N;

        #pragma omp simd
        for (int i = base; i < limit; i++) {
            C[i] = A[i] * B[i];
        }
    }

    printf("C[0] = %f, C[N-1] = %f\n", C[0], C[N-1]);

    free(A); free(B); free(C);
    return 0;
}
```